

FINAL PROJECT

Monthly Attendance Mailer

One Actor. Reads Excel from Drive, emails parents.

Agentic AI Workshop

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

All code is in Section 9 of this document.

Sample data: Monthly_Attendance_Sample.xlsx

No Google Cloud project. No API keys for Drive.

Monthly Attendance Mailer

One Apify Actor. It reads an Excel file from your Google Drive, works out each student's attendance percentage, and emails the parents of anyone below your threshold.

No Google Cloud project. No API keys for Drive. No OAuth. The file is reached through an ordinary Drive share link.

What you need

- A Google account, personal rather than institutional
- An Apify account, free plan is enough
- Ten minutes

The two files you were given

File	What it is
<code>Monthly_Attendance_Sample.xlsx</code>	Sample data. Fifteen invented students, two tabs.
This PDF	Every instruction, and all three code files, below.

ABOUT COPYING CODE OUT OF THIS PDF

The code in Section 9 is formatted so that no line wraps, which makes it safe to copy from here.

Even so, after pasting each file, check that the last line matches what is printed here. If a paste looks broken, the usual cause is a line that split in two.

1 • Put the Excel file on Google Drive

1. Go to **drive.google.com** and upload `Monthly_Attendance_Sample.xlsx`
2. Right-click the file, choose **Share**
3. Under **General access**, change *Restricted* to **Anyone with the link**
4. Click **Copy link**

The link looks like this:

```
https://drive.google.com/file/d/1AbCdEfGhIjKlMnOpQrSt/view?usp=sharing
```

Keep it. That whole link goes into the Actor.

LEAVE IT AS .XLSX

Do not convert the file to Google Sheets. The Actor reads Excel directly.

THIS IS FINE FOR THE SAMPLE, NOT FOR A REAL CLASS

"Anyone with the link" means exactly that. The URL needs no login.

The sample data is invented, so it does not matter. Before you use your own class list, talk to your department about where that file should live. It contains every student's name and every parent's email address.

2 · Create the Actor

1. In the Apify Console go to **Actors** → **Develop new** → **Create new Actor**
2. Work through the prompts: choose **Not sure**, then **JavaScript**, then the lightest template offered, then **host on Apify**
3. You land in the web code editor with a file tree

Now replace three files with the code from Section 9:

Open this	Do this
<code>src/main.js</code>	Select all, delete, paste 9.1
<code>.actor/input_schema.json</code>	Select all, delete, paste 9.2
<code>package.json</code>	Select all, delete, paste 9.3

PACKAGE.JSON MATTERS

It adds the two libraries the Actor needs: `xlsx` to read the Excel file, and `nodemailer` to send the mail. Skip it and the build fails.

3 • Create a Gmail app password

Gmail no longer accepts your ordinary password over SMTP.

1. Go to **myaccount.google.com/security**
2. Turn on **2-Step Verification** if it is off. App passwords do not exist without it.
3. Go to **myaccount.google.com/apppasswords**
4. Name it **Apify Attendance Mailer**, click **Create**
5. Copy the **16-character code**. Google shows it once.

USE A PERSONAL GMAIL

Institutional accounts often have app passwords disabled by the administrator. If the app passwords page shows nothing, that is why.

Also: treat this code like a password. Anyone holding it has full send access to your account. Revoke it from the same page when you no longer need it.

4 · Add the credentials, then build

On the Actor page go to **Settings** → **Environment variables**.

Add these. **Tick "Secret" on each one.**

Name	Value
<code>GMAIL_USER</code>	your.name@gmail.com
<code>GMAIL_APP_PASSWORD</code>	the 16-character code
<code>GEMINI_API_KEY</code>	only if you turn <code>useAI</code> on

Then click **Build**.

BUILD AFTER ADDING, NOT BEFORE

Environment variables are applied at build time. Add them and run without rebuilding, and the Actor reports them as missing.

5 · First run, preview only

Go to the **Input** tab.

Field	Value
Google Drive link	paste your link from Section 1
Attendance threshold	75
Preview or send	Preview only — leave it
Memory	512 MB

Click **Start**.

What you should see

With the sample data, exactly this:

```
Read 15 rows.
6 below 75%. 1 skipped.
  SKIPPED S015: days_present is blank
```

Then six drafted messages. Open the **Dataset** tab and read them all.

ID	Student	Attendance	Severity
S003	Chetan Deshmukh	45.8%	critical
S004	Deepa Joshi	70.8%	early warning
S006	Farhan Shaikh	33.3%	critical
S007	Gauri Naik	66.7%	early warning
S011	Kavya Pawar	58.3%	serious
S013	Manasi Thorat	54.2%	serious

If your numbers differ from these, stop and find out why before sending anything.

WHY S015 IS SKIPPED

Omkar Chavan has a blank `days_present` . The Actor skips the row and logs why, rather than reading blank as zero and telling a parent their child attended nothing.

Rows are also skipped when `total_days` is zero, when days present exceeds total days, or when the email address has no @.

6 • First live send, redirected to you

Every address in the sample ends in `.invalid`, a reserved domain that can never deliver anywhere. To see real mail, redirect it to yourself.

Field	Value
Redirect ALL mail to this address	your.name@gmail.com
Preview or send	Send

Click **Start**. All six arrive in **your** inbox, with subject lines like:

```
[TEST - for parent.s004@example.invalid] Attendance update - Deepa Joshi - September 2026
```

Read one properly, as a parent would. It reads differently in an inbox than in a table.

7 · Sending to real parents

Two things must both be true. This is deliberate.

1. **Redirect address must be empty**
2. **"I intend to email the real addresses" must be ticked**

Set only one and the Actor refuses, and tells you which is missing.

WHY TWO CONDITIONS

A single checkbox gets ticked without thinking. A single redirect field gets cleared by someone copying a configuration.

The failure here is not an error message. It is a parent being told their child missed fourteen classes when they missed four. You cannot unsend that.

Before you do this with a real class

Get your HoD to approve the wording, the threshold and the frequency. Contacting parents is a formal communication.

Check the email column is current. An out-of-date address means either nothing arrives, or a stranger reads about somebody's child.

Verify the spreadsheet. A marking error becomes a confidently worded wrong email. The Actor cannot know your data is wrong.

Tell students the system exists. A student who knows a parent will be contacted at 75% can come and talk to you first. That is the point — the alert should start a conversation, not be a surprise.

8 • Run it automatically every month

Actor page → **Schedules** → **Create schedule**

When	Cron
1st of the month, 9 am	0 9 1 * *
Last Friday, 5 pm	0 17 * * 5

SET THE TIMEZONE

Schedules default to **UTC**. 0 9 1 * * on UTC fires at **2.30 pm IST**.

Change the timezone to **Asia/Kolkata**, then check the **Next runs** preview before saving.

An Actor must have been run at least once before it can be scheduled.

9 • The code

9.1 `src/main.js`

```

import { Actor } from 'apify';
import * as XLSX from 'xlsx';
import nodemailer from 'nodemailer';

await Actor.init();

// =====
// INPUT
// =====
const input = await Actor.getInput() ?? {};

const fileSource    = String(input.fileSource ?? '').trim();
const threshold     = Number(input.thresholdPercent ?? 75);
const courseName    = String(input.courseName ?? 'this course').trim();
const teacherName   = String(input.teacherName ?? 'Your instructor').trim();
const monthName     = String(input.monthName ?? 'this month').trim();
const sendMode      = String(input.sendMode ?? 'preview').trim();
const overrideTo    = String(input.testRecipientOverride ?? '').trim();
const allowLive     = input.allowLiveSend === true;
const useAI         = input.useAI === true;
const delayMs       = Number(input.delayMs ?? 1500);

const model = String(input.model ?? '')
    .trim()
    .replace(/^models[/]/, '')
    .replace(/[^a-zA-Z0-9._-]/g, '')
    || 'gemini-2.5-flash';

if (!fileSource) {
    throw new Error('fileSource is empty. Paste your Google Drive share '
        + 'link, or a published CSV URL, into the Input tab.');
}

// =====
// CREDENTIALS
// =====
const GM_USER = process.env.GMAIL_USER;
const GM_PASS = process.env.GMAIL_APP_PASSWORD;
const API_KEY = process.env.GEMINI_API_KEY;

if (sendMode === 'send' && (!GM_USER || !GM_PASS)) {
    throw new Error('sendMode is "send" but GMAIL_USER and '
        + 'GMAIL_APP_PASSWORD are not set. Add them in Settings as '
        + 'SECRET environment variables, then REBUILD.');
}

if (useAI && !API_KEY) {
    throw new Error('useAI is on but GEMINI_API_KEY is not set. '
        + 'Add it in Settings, then REBUILD. Or turn useAI off.');
}

// ---- the safety gate ----
// Live sending needs the override cleared AND an explicit tick.
if (sendMode === 'send' && !overrideTo && !allowLive) {
    throw new Error('Refusing to send. testRecipientOverride is empty '
        + 'and allowLiveSend is false. Either set the override to your '
        + 'own address, or tick allowLiveSend to confirm you intend to '
        + 'email the real addresses in the file.');
}

```

```

if (overrideTo) {
  console.log('OVERRIDE ACTIVE - all mail goes to ' + overrideTo);
}

function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

// =====
// 1. READ THE FILE FROM GOOGLE DRIVE
// =====

// Accepts a Drive share link, a bare file ID, or a published CSV URL.
function buildDownloadUrl(src) {
  if (src.indexOf('output=csv') !== -1) {
    return { url: src, kind: 'csv' };
  }
  // .../file/d/FILE_ID/view or ...?id=FILE_ID
  let id = null;
  const m1 = src.match(/\/file\/d\/([a-zA-Z0-9_-]{20,})/);
  const m2 = src.match(/[?&]id=([a-zA-Z0-9_-]{20,})/);
  if (m1) id = m1[1];
  else if (m2) id = m2[1];
  else if (/^[a-zA-Z0-9_-]{20,}$/.test(src)) id = src;

  if (!id) {
    throw new Error('Could not find a Drive file ID in: ' + src
      + ' Expected a link like '
      + 'https://drive.google.com/file/d/FILE_ID/view');
  }
  return {
    url: 'https://drive.google.com/uc?export=download&id=' + id,
    kind: 'xlsx'
  };
}

const src = buildDownloadUrl(fileSource);
console.log('Fetching ' + src.kind + ' from Drive...');

const res = await fetch(src.url, { redirect: 'follow' });
if (!res.ok) {
  throw new Error('Download failed: HTTP ' + res.status
    + '. Check the file is shared as "Anyone with the link".');
}

let rows;

if (src.kind === 'csv') {
  const text = await res.text();
  const wb = XLSX.read(text, { type: 'string' });
  rows = XLSX.utils.sheet_to_json(wb.Sheets[wb.SheetNames[0]]);
} else {
  const buf = Buffer.from(await res.arrayBuffer());
  // A Drive permission page is HTML, not a spreadsheet.
  if (buf.slice(0, 5).toString().indexOf('<') === 0) {
    throw new Error('Drive returned a web page, not a file. The file '
      + 'is probably not shared. Open it in Drive, click Share, '
      + 'and set General access to "Anyone with the link".');
  }
}

```

```

    const wb = XLSX.read(buf, { type: 'buffer' });
    rows = XLSX.utils.sheet_to_json(wb.Sheets[wb.SheetNames[0]]);
  }

  console.log('Read ' + rows.length + ' rows.');
```

if (rows.length === 0) {
 throw new Error('The file has no data rows.');

}

// =====
// 2. VALIDATE AND COMPUTE
// Deterministic. No model involved. This must be exact.
// =====
const REQUIRED = ['student_id', 'student_name', 'days_present',
 'total_days', 'parent_email'];

const header = Object.keys(rows[0]);
const missing = REQUIRED.filter((c) => header.indexOf(c) === -1);
if (missing.length) {
 throw new Error('Missing column(s): ' + missing.join(', ')
 + '. Found: ' + header.join(', '));
}

const flagged = [];
const skipped = [];

for (const r of rows) {
 const present = Number(r.days_present);
 const total = Number(r.total_days);
 const email = String(r.parent_email ?? '').trim();

 if (r.days_present === undefined || r.days_present === ''
 || Number.isNaN(present)) {
 skipped.push({ id: r.student_id, why: 'days_present is blank' });
 continue;
 }
 if (!total || Number.isNaN(total)) {
 skipped.push({ id: r.student_id, why: 'total_days is zero' });
 continue;
 }
 if (present > total) {
 skipped.push({ id: r.student_id, why: 'present > total' });
 continue;
 }
 if (email.indexOf('@') === -1) {
 skipped.push({ id: r.student_id, why: 'parent_email malformed' });
 continue;
 }

 const pct = Number(((present / total) * 100).toFixed(1));
 if (pct >= threshold) continue;

 let severity;
 if (pct < 50) severity = 'critical';
 else if (pct < 65) severity = 'serious';
 else severity = 'early warning';

 flagged.push({
 student_id: String(r.student_id),

```

        student_name: String(r.student_name),
        parent_email: email,
        days_present: present,
        total_days: total,
        percentage: pct,
        severity
    });
}

console.log(flagged.length + ' below ' + threshold + '%. '
    + skipped.length + ' skipped.');
```

```

for (const s of skipped) {
    console.log(' SKIPPED ' + s.id + ': ' + s.why);
}

// =====
// 3. WRITE THE MESSAGE
// =====

function templateBody(s) {
    return 'Dear Parent,\n\n'
        + 'I am writing regarding ' + s.student_name
        + ' (ID ' + s.student_id + ') and their attendance in '
        + courseName + ' for ' + monthName + '.\n\n'
        + 'They attended ' + s.days_present + ' of ' + s.total_days
        + ' working days, which is ' + s.percentage + ' percent. '
        + 'Our expectation for this course is ' + threshold
        + ' percent.\n\n'
        + 'Attendance matters a great deal in this subject because each '
        + 'topic builds on the previous one, and missed sessions are '
        + 'difficult to recover from independently.\n\n'
        + 'If something is affecting their attendance that I should know '
        + 'about, please reply to this email or come and meet me. I would '
        + 'rather understand the situation than simply record the '
        + 'absence.\n\n'
        + 'Warm regards,\n' + teacherName;
}

const API_BASE = 'https://generativelanguage.googleapis.com/v1beta/models';

function endpoint() {
    return API_BASE + '/' + model + ':generateContent?key=' + API_KEY;
}

async function aiBody(s) {
    const prompt = 'You are a college professor writing to the parent of '
        + 'a student about attendance.\n\n'
        + 'STUDENT: ' + s.student_name + ' (ID ' + s.student_id + ')\n'
        + 'COURSE: ' + courseName + '\n'
        + 'MONTH: ' + monthName + '\n'
        + 'ATTENDANCE: ' + s.days_present + ' of ' + s.total_days
        + ' days = ' + s.percentage + ' percent\n'
        + 'THRESHOLD: ' + threshold + ' percent\n'
        + 'SEVERITY: ' + s.severity + '\n\n'
        + 'Write a short email body.\n\n'
        + 'Rules:\n'
        + '- Address the parent respectfully\n'
        + '- State the attendance figures plainly and accurately. Do NOT '
        + 'round, exaggerate, or add figures you were not given.\n'
```



```

+ '- Explain briefly why attendance matters for this course\n'
+ '- Invite them to reply or meet. Do not demand.\n'
+ '- Do NOT threaten, and do NOT mention marks, grades or exam '
+ 'eligibility. You were not given that information.\n'
+ '- 5 to 7 sentences\n'
+ '- Sign as ' + teacherName + '\n\n'
+ 'Write only the email body. No subject line. No preamble.';

const r = await fetch(endpoint(), {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    contents: [{ parts: [{ text: prompt }] }],
    generationConfig: { temperature: 0.5, maxOutputTokens: 1024 }
  })
});

if (!r.ok) {
  const b = await r.text();
  console.log(' AI failed (' + r.status + '), using template. '
    + b.slice(0, 120));
  return templateBody(s);
}

const d = await r.json();
const text = d?.candidates?.[0]?.content?.parts?.[0]?.text;
if (!text) {
  console.log(' AI returned nothing, using template. ');
  return templateBody(s);
}

// The drafted message must contain the real figures.
const t = text.trim();
if (t.indexOf(String(s.days_present)) === -1
  && t.indexOf(String(s.percentage)) === -1) {
  console.log(' AI draft omitted the figures, using template. ');
  return templateBody(s);
}
return t;
}

const messages = [];

for (let i = 0; i < flagged.length; i++) {
  const s = flagged[i];
  console.log('[' + (i + 1) + '/' + flagged.length + ']' + ' '
    + s.student_name + ' ' + s.percentage + '% (' + s.severity + ')');

  let body;
  if (useAI) {
    body = await aiBody(s);
    if (i < flagged.length - 1) await sleep(4500);
  } else {
    body = templateBody(s);
  }

  messages.push({
    student_id: s.student_id,
    student_name: s.student_name,

```

```

        parent_email: s.parent_email,
        days_present: s.days_present,
        total_days: s.total_days,
        percentage: s.percentage,
        severity: s.severity,
        subject: 'Attendance update - ' + s.student_name
            + ' - ' + monthName,
        body,
        status: 'drafted',
        delivered_to: null,
        error: null
    });
}

// =====
// 4. PREVIEW OR SEND
// =====
if (sendMode === 'send') {
    console.log('');
    console.log('*** SEND MODE - mail is going out ***');

    const transporter = nodemailer.createTransport({
        host: 'smtp.gmail.com',
        port: 465,
        secure: true,
        auth: { user: GM_USER, pass: GM_PASS }
    });

    try {
        await transporter.verify();
        console.log('SMTP connection verified.');
```

```
    } catch (err) {
        throw new Error('Gmail login failed: ' + err.message
            + ' Check GMAIL_USER, and that GMAIL_APP_PASSWORD is a '
            + '16-character app password with 2-Step Verification on.');
```

```
    }

    for (const m of messages) {
        const to = overrideTo || m.parent_email;
        const subject = overrideTo
            ? '[TEST - for ' + m.parent_email + ']' + m.subject
            : m.subject;

        try {
            await transporter.sendMail({
                from: '"' + teacherName + '" <' + GM_USER + '>',
                to,
                subject,
                text: m.body
            });
            m.status = 'sent';
            m.delivered_to = to;
            console.log(' sent to ' + to + ' re ' + m.student_name
                + (overrideTo ? ' [REDIRECTED]' : ''));
        } catch (err) {
            m.status = 'send_failed';
            m.error = err.message;
            console.log(' FAILED ' + m.student_name + ': ' + err.message);
        }
    }
}

```

```

        await sleep(delayMs);
    }
} else {
    console.log('');
    console.log('Preview mode. Nothing was sent.');
    console.log('Read the dataset, then set sendMode to "send".');
}

await Actor.pushData(messages);

console.log('');
console.log('=== SUMMARY ===');
console.log('Rows read      : ' + rows.length);
console.log('Skipped          : ' + skipped.length);
console.log('Below ' + threshold + '%      : ' + flagged.length);
console.log('Sent              : '
    + messages.filter((m) => m.status === 'sent').length);
console.log('Mode              : ' + sendMode
    + (overrideTo ? ' (redirected)' : ''));

await Actor.exit();

```

9.2 `.actor/input_schema.json`

```

{
  "title": "Monthly Attendance Mailer",
  "type": "object",
  "schemaVersion": 1,
  "properties": {
    "fileSource": {
      "title": "Google Drive link to the attendance file",
      "description": "Paste the Drive share link for your .xlsx file. The file must be
shared as 'Anyone with the link'. A published CSV URL also works.",
      "type": "string",
      "editor": "textfield",
      "prefill": "https://drive.google.com/file/d/PASTE_YOUR_FILE_ID_HERE/view?usp=sharing"
    },
    "thresholdPercent": {
      "title": "Attendance threshold (%)",
      "description": "Students below this percentage have their parent contacted.",
      "type": "integer",
      "editor": "number",
      "default": 75
    },
    "monthName": {
      "title": "Month this covers",
      "description": "Appears in the subject line and the message.",
      "type": "string",
      "editor": "textfield",
      "default": "September 2026"
    },
    "courseName": {
      "title": "Course name",
      "description": "Appears in the message body.",
      "type": "string",
      "editor": "textfield",
      "default": "Database Management Systems (CS204)"
    },
    "teacherName": {
      "title": "Your name",
      "description": "Used in the signature and the From line.",
      "type": "string",
      "editor": "textfield",
      "default": "Prof. A. Sharma"
    },
    "sendMode": {
      "title": "Preview or send",
      "description": "Leave on Preview until you have read a full run in the dataset.",
      "type": "string",
      "editor": "select",
      "enum": ["preview", "send"],
      "enumTitles": [
        "Preview only - write to dataset, send nothing",
        "Send - mail actually goes out"
      ],
      "default": "preview"
    },
    "testRecipientOverride": {
      "title": "Redirect ALL mail to this address",
      "description": "Strongly recommended for your first send. Every message goes here
instead of the parent, with the intended recipient shown in the subject.",
      "type": "string",

```

```

    "editor": "textfield",
    "default": ""
  },
  "allowLiveSend": {
    "title": "I intend to email the real addresses in the file",
    "description": "Only tick this once the override is cleared AND you have read a full
preview run.",
    "type": "boolean",
    "default": false
  },
  "useAI": {
    "title": "Use Gemini to write the message",
    "description": "Off uses a fixed template - reliable, free, no API key needed. On
personalises each message. Falls back to the template if the AI fails.",
    "type": "boolean",
    "default": false
  },
  "model": {
    "title": "Gemini model",
    "description": "Only used when useAI is on. Type the identifier, do not paste it.",
    "type": "string",
    "editor": "textfield",
    "default": "gemini-2.5-flash"
  },
  "delayMs": {
    "title": "Delay between emails (ms)",
    "description": "Gmail throttles rapid sending. 1500 is safe.",
    "type": "integer",
    "editor": "number",
    "default": 1500
  }
},
"required": ["fileSource"]
}

```

9.3 package.json

```

{
  "name": "attendance-mailer",
  "version": "0.0.1",
  "type": "module",
  "description": "Reads a monthly attendance file from Google Drive and mails parents of
students below a threshold.",
  "dependencies": {
    "apify": "^3.2.0",
    "xlsx": "^0.18.5",
    "nodemailer": "^6.9.14"
  },
  "scripts": {
    "start": "node src/main.js"
  }
}

```

10 • Using your own class

Keep these five column names **exactly**, lowercase with underscores:

```
student_id
student_name
days_present
total_days
parent_email
```

Everything else can change. Add as many rows as you like. The Actor tells you if a column is missing.

The AI switch

`useAI` is **off** by default, which uses a fixed template.

Off is reliable, free, needs no API key, produces the same message every month, and cannot invent a figure.

On has Gemini write each message individually — warmer and more specific. Needs `GEMINI_API_KEY`.

Even with AI on, the Actor checks that the drafted message contains the real attendance figures, and falls back to the template if it does not. A parent mailer that quietly stops working is worse than one that sends a plainer email.

For a monthly attendance note, off is arguably the right setting. Knowing when not to use the AI is part of the skill.

11 • Troubleshooting

Message	Cause	Fix
Drive returned a web page, not a file	File is not shared	Share → Anyone with the link
Could not find a Drive file ID	Wrong kind of link	Use the <code>/file/d/.../view</code> link
GMAIL_USER and GMAIL_APP_PASSWORD are not set	Added but not rebuilt	Rebuild
Gmail login failed	Not a 16-character app password, or 2-Step Verification off, or an institutional account	Section 3
Missing column(s)	Header names do not match	Section 10
Refusing to send	Only one of the two send conditions is set	Section 7
Build fails immediately	<code>package.json</code> not replaced	Section 2
Cannot use import statement	<code>package.json</code> missing <code>"type": "module"</code>	Paste 9.3 again
Runs but sends nothing	Still on Preview	That is the default, on purpose
Mail lands in spam	Normal for bulk mail from a personal Gmail	For real use, send from an institutional address via your IT department
Nothing sends from the lab	Campus firewall blocking port 465	Ask your network team. Preview mode still works.

CHECK THIS BEFORE A LIVE DEMO

Outbound port 465 is a common spam vector and is blocked on many campus networks.

Test one redirected send from the lab machine, on the lab network, before you rely on it in front of anyone.